DATA ENGINEERING CAPSTONE

Sephora Reviews Analytics Pipeline

How 1.09 million product reviews travel from six raw CSV files to a live dashboard — every stage, what it changes, and why it is built that way.

1,093,371
REVIEWS IN THE WAREHOUSE

8,494
PRODUCTS

503,216
REVIEWERS

15
AIRFLOW TASKS

11
ANALYTICS VIEWS

51
PASSING TESTS

#	Stage	What it produces
1	Source data	6 CSVs — the columns and what they mean
2	Explore	14 profiling checks, measured before any rule was written
3	Clean	`data/processed/` — 1,040 duplicates removed
4	OLTP	`raw` → `3nf` (9 tables) → `staging`
5	ETL	extract → transform → reconcile → quality → load
6	Warehouse	Star schema: 5 dimensions + `fact_reviews`
7	Quality gate	7 check types, two severities
8	Airflow	15 tasks, 3 load modes, watermark incremental
9	Dashboard	One page, 5 business questions, live connection

Samekshya Baniya · PostgreSQL 16 · Python 3.13 · Apache Airflow 3.3 · Streamlit · Every figure in this document was measured from the running system, not estimated.

STAGE 1 The source data

Two kinds of file at two different grains, from the Kaggle dataset *Sephora Products and Skincare Reviews*. Nothing here has been touched by hand.

product_info.csv
8,494 rows × 27 columns
One row per product

+

reviews_*.csv (× 5)
1,094,411 rows total
One row per review

=

≈ 550 MB
Gitignored. `data/README.md` says where to get them

product_info.csv — all 27 columns

Column	Type	Meaning and fate in the pipeline
`product_id`	text	PK Unique — 0 duplicates. Survives to `dim_product`
`product_name`	text	Carried through to `dim_product`
`brand_id`	integer	FK Strictly 1:1 with `brand_name` — becomes the `brand` table
`brand_name`	text	Moved out to `3nf.brand`; removed from product (redundancy)
`loves_count`	integer	Wishlist adds. Drives BQ2 — recorded *before* purchase
`rating`	numeric	Site-displayed average. **Not used** — the warehouse recomputes from reviews
`reviews`	integer	Site-displayed count. Not used, for the same reason
`size`	text	Free text ("1.7 oz / 50 mL"). Kept on `dim_product`
`variation_type` / `_value` / `_desc`	text	Shade or size variants. Dropped at raw → 3NF (out of scope)
`ingredients`	text	Long free text. Dropped at raw → 3NF
`price_usd`	numeric	Drives BQ3. Banded into `price_band` once, in the ETL
`value_price_usd`, `sale_price_usd`	numeric	Sparse. Dropped at raw → 3NF
`limited_edition`, `new`, `online_only`, `out_of_stock`, `sephora_exclusive`	boolean	Five flags. All kept on `dim_product`
`highlights`	text	Stringified list, 112 tags. **Measured, then deliberately descoped** — a genuine many-to-many (D3)
`primary_category`	text	Always `Skincare` for reviewed products (D16)
`secondary_category`	text	The level that actually varies — all category analysis runs here
`tertiary_category`	text	Third level. Kept, flattened onto `dim_product`
`child_count`, `child_max_price`, `child_min_price`	int/num	Variant rollups. Dropped at raw → 3NF

reviews_*.csv — all columns

Column	Type	Meaning and fate in the pipeline
`source_row_id`	bigint	idempotency key The CSV row index. **Restarts at 0 in each file** — safe only when paired with `product_id` (D13)
`author_id`	text	FK 503,216 distinct. Becomes `3nf.author` / `dim_customer` — **identity only**
`product_id`	text	FK 0 orphans — every review resolves to a real product
`rating`	smallint	1–5. The central measure; `CHECK`-constrained in the warehouse

`is_recommended`	boolean	≈15% unanswered. Left NULL, never guessed
`helpfulness`	numeric	NULL ⇔ `total_feedback_count = 0`. **Undefined, not missing** (D5)
`total_feedback_count`	integer	Verified: pos + neg = total on all 1.09M rows, 0 violations
`total_pos_feedback_count`	integer	As above
`total_neg_feedback_count`	integer	As above
`submission_time`	date	2008-08-28 → 2023-03-21. **Every timestamp is midnight**, so date grain loses nothing
`review_text`	text	350 MB. **Stops at the OLTP boundary**; only its length goes to the warehouse (D6)
`review_title`	text	Stops at the OLTP boundary
`skin_tone`, `skin_type`, `eye_color`, `hair_color`	text	Self-reported, **per review**, and they drift. This is the project's key modelling problem — see Stage 6
`product_name`, `brand_name`, `price_usd`	text/num	Repeated on every review row; agree 100% with the catalogue. Dropped at staging
`source_file`	text	Added by `ingest.py` for traceability
`review_length`	integer	Derived by `clean.py` before the text is discarded

> **Why the catalogue is bigger than the reviewed set**
>
> 8,494 products exist but only **2,351** carry reviews — and all of them are Skincare. The catalogue was scraped in full; reviews only for skincare. Every conclusion in this project is therefore about skincare, not about Sephora as a whole.

STAGE 2 Explore — measure before deciding

`explore.py` runs 14 profiling checks and writes nothing. Every cleaning rule and design decision downstream cites a number from here, so no rule is a guess.

What was measured	Result
Duplicate `product_id`	0
`brand_id` ↔ `brand_name`	1:1, 304
Category triples	174
Orphan reviews	0
Non-midnight timestamps	0
Dupes on (author, product, date)	1,040
Dupes on (author, product)	5,525
Collisions on (row_id, product)	0

What was measured	Result
Authors with a drifting profile	22,503
...as a share of authors	4.47%
Reviews they wrote	149,788
...as a share of reviews	13.69%
Distinct profile combinations	2,003
Feedback-count violations	0
helpfulness ⇔ 0-votes mismatches	0
2023 rows (held for incremental)	49,531

> **The measurement that changed the design**
> **22,503 reviewers (4.47%) gave more than one distinct value** for skin tone, skin type, eye colour or hair colour across their reviews — affecting **149,788 reviews, 13.69% of the dataset**. People's self-reported skin type is not fixed; it is answered per review and it drifts. This single number is why a conventional customer dimension was rejected. See Stage 6.

STAGE 3 Clean

`clean.py` reads `data/raw/` and writes `data/processed/`. Structural cleaning only — and it **never drops a column**.

Rule	Rows affected	Why
Remove exact duplicates	1,040	Keyed on (author_id, product_id, submission_time). The looser key would have removed **5,525** — the 4,485 difference is legitimate re-reviews, months apart (D4)
Trim whitespace	1,046,825 cells	1,030,753 in reviews + 16,072 in products
`'Grey'` → `'gray'`	4,859	Same colour, two spellings — would have split one eye-colour group in two
`'notSureST'` → NULL	70	A "not sure" placeholder is not a skin tone
Derive `review_length`	all	Computed while the text still exists, because it is dropped at Stage 4
Assert `product_id` unique	—	Raises and halts rather than proceeding on a broken key
Assert brand 1:1	—	As above

Result: 1,094,411 → **1,093,371** reviews; 8,494 → 8,494 products; 0 unparseable dates. Runs in ~24 seconds.

> **Why cleaning drops rows but never columns (D14)**
> Mixing row-cleaning and column-dropping in one step makes it impossible to tell later which removal was a *cleaning rule* and which was a *scope decision*. Every source column therefore reaches the `raw` schema, and columns are trimmed exactly once, explicitly, at the raw → 3NF boundary.

STAGE 4 The OLTP database

`sephora_oltp` has three schemas, each with exactly one job. This is the stage that demonstrates normalisation — and then deliberately undoes some of it.

raw
1:1 mirror of the cleaned CSVs. Every source column kept. Loaded by `COPY` in one transaction.

→

3nf
9 tables, every foreign key enforced by Postgres. Redundancy removed.

→

staging
De-normalised again on purpose, so the ETL read is a plain `SELECT`.

The 3NF model — 9 tables, all live counts

Table	Rows	Key and purpose
`brand`	304	brand_id PK `brand_name` UNIQUE — the 1:1 was verified, not assumed
`category`	174	serial PK UNIQUE on the **full triple**. Not a hierarchy — one secondary category appears under up to 7 primaries (D1)
`product`	8,494	FK → brand, category
`author`	503,216	author_id PK **No descriptive attributes at all** (D2)
`skin_tone`	13	Lookup table
`skin_type`	4	Lookup table
`eye_color`	5	Lookup table
`hair_color`	7	Lookup table
`review`	1,093,371	FK → author, product + the 4 lookups The four reviewer attributes live **here**, at review grain

What changes at each boundary

Boundary	What happens
CSV → `raw`	`COPY`, truncate-and-reload inside one transaction. Loaded counts asserted against what `clean.py` reported — a mismatch raises
`raw` → `3nf`	**Columns are dropped here and only here**: `ingredients`, `highlights`, the variation and child-price columns, and the review columns that duplicate the catalogue. Brand and category are extracted into their own tables. Missing reviewer attributes become **NULL foreign keys**
`3nf` → `staging`	Dimension attributes pre-joined back on. `review_text` and `review_title` are **left behind permanently** (D6). NULL → the string `'Unknown'` happens **here**, not in 3NF

> **Why normalise and then de-normalise**
> The two layers answer different questions. `3nf` answers "is this data internally consistent?" — enforced by foreign keys, with a missing value as a NULL FK. `staging` answers "what is the average rating by brand?" — optimised for reads, with a missing value as the string `'Unknown'`. Building both is what demonstrates the difference between an OLTP model and a dimensional one, rather than asserting it.

Verified end to end: 0 row gap on both products and reviews across all three layers, and all 6 integrity checks return 0.

STAGE 5 ETL — OLTP to warehouse

Five modules in `etl/`, one responsibility each. The same functions run from the command line and from Airflow — there is no second implementation.

extract Reads `staging`. Three modes. → **transform** Natural keys → surrogate keys. Returns (rows, drops-by-reason). → **reconcile** in = out + dropped, **or raise**. → **quality** A gate. Never repairs. → **load** `ON CONFLICT DO NOTHING` everywhere.

The three load modes (D17)

Mode	Bound	Rows	Purpose
`full`	No date bound	1,093,371	Everything, in one command
`historical`	`< 2023-01-01`	1,043,868	The demo baseline — deliberately holds 2023 back
`incremental`	`> watermark`	49,503	Only what is new

The old design had a `--full-reload` flag that stopped at 2023 while calling itself full, and no way to load everything at once. Three honest names replaced it.

The watermark

`MAX(submission_date)` read **from the fact table itself**, so it cannot disagree with the data it describes. Two details that matter:

Captured before any write	Read at the start of the extract task, so it cannot advance past rows the same run is loading
Strictly `>`, not `>=`	A current watermark extracts **0** rows, rather than re-offering the same day forever

Reconciliation — the rule that makes silent loss impossible (D19)

Every row entering a stage must leave it either transformed or dropped **for a named reason**. Reasons are reported even when zero, so "nothing was dropped" is distinguishable from "this was never checked". An unexplained gap raises `ReconciliationError` and fails the run.

```
fact_reviews: row counts do not balance. extracted=100, transformed=90,
dropped=5 (accounted=95), UNEXPLAINED=5.
```

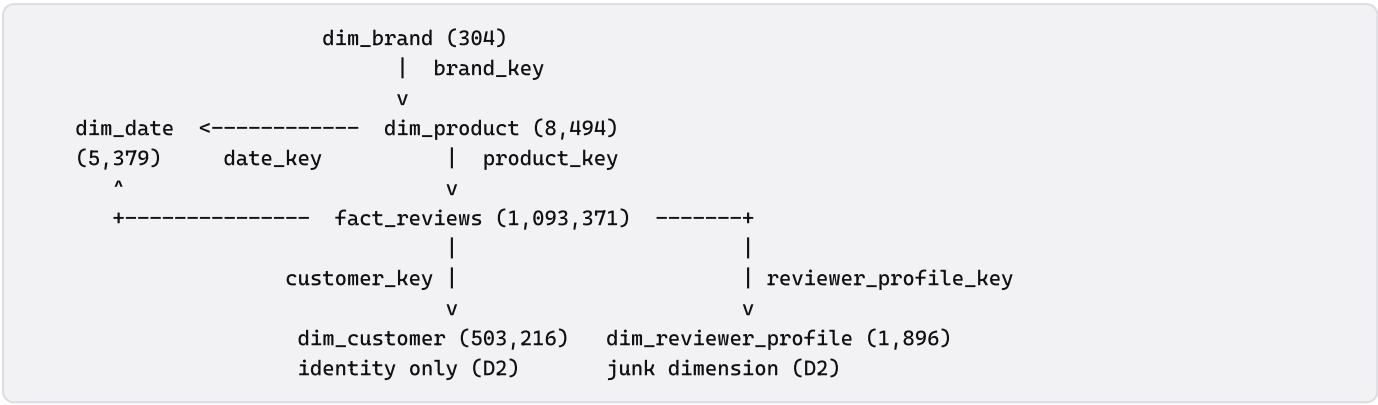
The four named drop reasons: unresolved product, unresolved customer, unresolved reviewer profile, out-of-range date.

Idempotency — measured, not asserted

Scenario	Offered	Inserted
First historical load	1,043,868	1,043,868
Re-run of the same load	1,043,868	**0**
Incremental, watermark 2022-12-31	49,503	49,503
Re-run incremental, watermark current	**0**	0

STAGE 6 The warehouse — star schema

`sephora_dw`, schema `dw`. Grain of the fact table: **one row per review.**



Dimensions

Table	Rows	Key columns and notes
`dim_date`	5,379	date_key INT `YYYYMMDD`. Range **derived from the data** with 30 days padding, not hardcoded (D12)
`dim_brand`	304	brand_key serial `brand_id` UNIQUE
`dim_product`	8,494	product_key serial FK → dim_brand All three category levels flattened on; price, `price_band`, size, loves, 5 flags
`dim_customer`	503,216	customer_key serial **Shopper identity only** — no descriptive attributes
`dim_reviewer_profile`	1,896	junk dimension UNIQUE on (skin_tone, skin_type, eye_color, hair_color)

fact_reviews

Column group	Contents
Surrogate key	`review_key serial`
Idempotency key	UNIQUE (source_row_id, product_id) — verified to collide 0 times across all 1,094,411 source rows (D13)
Foreign keys	`product_key`, `customer_key`, `reviewer_profile_key`, `date_key` — all four indexed
Measures	`rating` (CHECK 1–5), `is_recommended`, `helpfulness`, three feedback counts, `review_length`
Watermark	`submission_date`

The decision worth defending — why a junk dimension (D2)

The obvious design puts skin tone, skin type, eye colour and hair colour on a customer dimension keyed uniquely on the reviewer. Measured against this data, that breaks: **22,503 reviewers (4.47%) report more than one distinct value**, and because prolific reviewers are over-represented among them, those reviewers account for **149,788 reviews — 13.69% of the dataset**.

One row per customer forces one profile per person, so roughly **one review in seven** would carry a profile the reviewer never gave on that review — silently, with no constraint violation and nothing downstream to notice. It would have quietly corrupted BQ4 specifically. The fix: `dim_customer` keeps identity only, and a **1,896-row junk dimension** holds the four attributes at the grain they were actually recorded.

No brand_key on the fact table (D11)

Brand is functionally determined by product, so a copy on the fact adds no information and creates a way for the two to disagree. It lives on `dim_product` only.

STAGE 7 The quality gate

`etl/quality.py` is a **gate, not a fixer**. It raises `DataQualityError`; it never repairs data. Repairing at load time would hide the fact that something upstream is wrong.

The seven checks

Check	Severity	What it catches
`check_row_count`	**hard failure**	An empty or truncated batch
`check_no_null_keys`	**hard failure**	A null surrogate key — named by column, instead of a bare Postgres NOT NULL violation mid-insert
`check_no_negative_values`	**hard failure**	A negative feedback count means the transform corrupted something
`check_value_range`	**hard failure**	Rating outside 1–5
`check_referential_integrity`	**hard failure**	A key absent from its dimension — before the FK rejects it
`check_unique_key`	**hard failure**	Duplicate (source_row_id, product_id)
`check_null_rate`	warning	A *shift* in null rate, e.g. `is_recommended` suddenly arriving 90% null

> **Why two severities and not all-fatal (D21)**
>
> All-fatal sounds rigorous and is a trap: it means the only checks worth writing are ones you would stop production for, so anything softer never gets written and the gate stays silent about things worth knowing. `is_recommended` is legitimately ~15% null and `helpfulness` is undefined wherever nobody voted — failing a run over that would be wrong, and saying nothing would be worse.
>
> Warnings are logged **before** the hard-failure raise, so a run that dies still leaves its warnings behind.

Behaviour

Hard failure	Halts **before any write**. In Airflow it raises `AirflowFailException`, which fails fast rather than burning the 2-retry budget — bad data will not become good on a second attempt
Warning	Logged, run continues
Empty batch	Gate skipped, run succeeds. An incremental run with nothing new is a clean no-op, not a crash

Proven by fault injection

The point is not that clean data passes — every run proves that. The point is that dirty data **fails**. A gate never observed to fail is indistinguishable from a gate that cannot fail. **15 tests** each start from one valid frame and break exactly one thing, so a failure names the defect. All 15 pass.

STAGE 8 Orchestration — the Airflow DAG

`sephora_dw_pipeline_staged` — **15 tasks, 21 edges**, Airflow 3.3.0, LocalExecutor. Load mode is chosen from the UI as an enum parameter.

Design point	Reason
Three dimension branches in parallel	They share no dependency. `product` waits on `brand` because it resolves `brand_key`
Fact split into 4 staged tasks	Each is independently retryable, a load failure does not re-hit the OLTP source, and the Graph view **names the stage** that failed instead of showing one red box
No row data in XCom	XCom is metadata storage. 503,216 customer rows through it would abuse the metadata database. Intermediate results go to `dw.stg_*` tables scoped by `batch_id = run_id`
Retry deletes its own batch first	A retry re-uses the same `run_id`, so staged rows are deleted before re-staging — otherwise a retry after a partial write silently doubles everything
Fact loads in 100,000-row chunks	The first full run was **SIGKILLed**: materialising 1,043,868 rows and then building a list of tuples meant two full copies resident at once (D15)

The bug worth presenting — a green run over an empty warehouse (D20 → D24)

`cleanup_staging` uses `trigger_rule="all_done"` so a failed run still clears its staging rows. Correct in itself — but it made cleanup the last **leaf** task, and Airflow derives a DAG run's state from its leaves:

`extract fails → cleanup still runs → cleanup succeeds → the only leaf is green → the DAG run reports SUCCESS`

A green run that loaded nothing is worse than a red one, because nobody investigates it. It was first fixed with a watcher task wired downstream of all 15 others — 15 of the DAG's 33 edges existed purely for failure reporting. It is now fixed by marking cleanup a **teardown**: Airflow excludes ignorable teardowns from run-state calculation, so `load_fact_from_staging` becomes the effective leaf and failure reaches it as `upstream_failed`. Same guarantee, 15 tasks and 21 edges instead of 16 and 33.

Verified by forcing a failure, not by reading the docs

Observation	Result
`extract_fact_to_staging`	failed (injected)
transform / quality / load_fact	`upstream_failed` — propagation reaches the leaf
`cleanup_staging`	**success** — the teardown still cleaned up

DAG run state	**FAILED** — a green cleanup beside a red run
Staging tables afterwards	All 6 at **0 rows**

That injection also exposed a latent race: cleanup waited only on `load_fact`, so a short-circuiting fact chain let it run before the dimension branches had staged, stranding 513,606 rows. Cleanup now waits on all four staging writers.

Measured runs

Run	Duration	Result
`historical`	134 s	All tasks green; 1,043,868 fact rows
`incremental`	22 s	All tasks green; +49,503 → 1,093,371 total

STAGE 9 The dashboard

Streamlit over `sephora_dw`. **One page**, five business questions, a live Postgres connection — not a static export.

#	Question	What the data says
BQ5	How do volume and rating trend over time?	Volume grew twelve years to a peak, then eased. Rating dipped to 4.21 in 2020 and recovered to 4.34
BQ1	Which brands rate best and worst?	MARA 4.86 → Topicals 3.66 — a spread of **1.20 stars**, the largest effect anywhere on the page
BQ1b	Which categories rate best?	Cleansers 4.344 → Self Tanners 4.080. Spread of 0.26 — **category matters far less than brand**
BQ3	Does price predict satisfaction?	**An inverted U.** 4.238 under $15 → **4.334 at $50–100** → 4.271 above $100. Agreement peaks in the **same band** (std dev 1.0996, widening to 1.1366 above $100), so $50–100 is the sweet spot on both measures
BQ2	Hype vs reality	The Ordinary Vitamin C 23%: **132,601 loves, 3.45 rating**. Loves are recorded before purchase, ratings after — the gap is marketing outperforming the product
BQ4	Do reviewer profile or review length matter?	Both barely. Skin type moves the average 0.04. Review length moves it almost not at all — but short reviews are **polarised**: 1★ share falls 8.15%→3.73% *and* 5★ share falls 67.35%→62.52%, so both tails shrink and cancel in the mean

Filters

Control	Binds into	Scope
Category (multiselect)	`secondary_category = ANY(%s)`	Brands, categories, hype, skin type, product explorer
Minimum reviews per brand	`WHERE review_count >= %s`	The brand chart
Brand (multiselect)	`brand_name = ANY(%s)`	Product explorer
Product name contains	`product_name ILIKE %s`	Product explorer
Refresh data	Clears the query cache	Whole page

> **Controls are query parameters, not dataframe filters (D23)**
> Every control above binds into the SQL and Postgres is asked again. A client-side filter over a preloaded frame would look identical on screen and be a different claim entirely — the tests assert this per control by moving one and requiring the corresponding caption to change.
>
> **The trend and price-band sections stay catalogue-wide** and are labelled *all categories*, because their views aggregate the category column away. A filter applied to them would do nothing while appearing to work.

Charts that had to change form

Most effects here are a tenth of a star, so axes must be truncated to show them — and **a truncated axis under bars misleads**, because bar length is read from zero. On the original price chart "Under $15" (4.2383) was drawn about six times shorter than "$50–100" (4.3335): a 2% difference rendered as 600%. Price became a line across the ordered bands; category and skin type became dot plots. A dot encodes *position*, so the same truncation is honest.

Review length became **small multiples**. The 1★ share (3–8%) and 5★ share (61–67%) are on different scales; on one axis the 1★ decline — half the finding — flattened into a strip along the baseline. A second y-axis would have invented a relationship rather than fixed one.

Why this is demonstrably live

Load `historical` mode, run the incremental DAG, press **Refresh data** — the review count moves from 1,043,868 to **1,093,371** and the watermark advances to 2023-03-21, on screen. A CSV export could not do that, and a screenshot certainly could not.

Every number in this document was measured from the running system on 2026-08-12. Reproduce them with `sql/validation/dashboard_checks.sql` — all 10 full-population views reconcile to `fact_reviews` at exactly 1,093,371, with 0 orphan keys and 0 duplicate idempotency keys.